

# Appunti AESO 2025/2026

Gabriele Fiorani

November 5, 2025

## 1 Livello dell'architettura

Il **Livello architetturale (Instruction Set Architecture - ISA)** definisce come il processore appare ai programmatori e ai compilatori.

- Non descrive **come** è costituito il processore (quello è il ruolo della microarchitettura).
- Si occupa invece di **cosa** il processore può fare e come il software può interagire con esso.

In pratica, è un **contratto tra hardware e software**: stabilisce le istruzioni disponibili, i registri e le modalità di accesso alla memoria.

### 1.1 Instruction Set Architecture - ISA

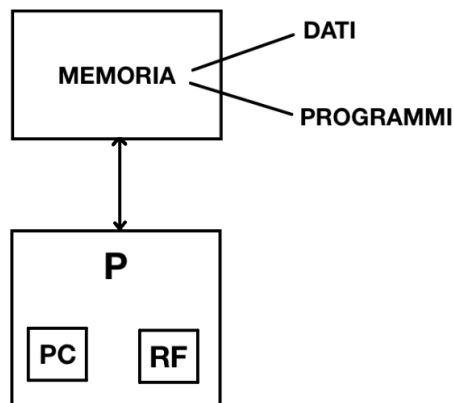
L'**ISA (Instruction Set Architecture)** è l'insieme di tutte le **istruzioni, tipi, registri, modi di indirizzamento e modalità operative** che un processore supporta. In pratica è ciò che il compilatore deve conoscere per generare codice eseguibile.

**Componenti principali:**

- **Set di istruzioni:** add, sub, load, branch...
- **Formato delle istruzioni:** struttura binaria, opcode, operandi
- **Dimensioni registri:** dimensione dei dati (registri) su cui l'architettura lavora (32 bit, 64 bit)
- **Tipi di dati supportati:** interi, floating point, vettori
- **Registri visibili al programmatore:** r0, r1, ...
- **Modi di indirizzamento:** diretto, indiretto, indicizzato, immediato

La lunghezza dell'istruzione dipende quindi dal design del ISA e non dalla dimensione dei registri.

## 1.2 Ciclo di esecuzione della CPU



```
P == {  
    while true do:  
        Fetch(PC);  
        Decode(Istruzione);  
        Execute(Istruzione);  
        Memory(Istruzione);  
        WriteBack(Istruzione);  
        PC = PC + 4;  
    }
```

- **Fetch:** carica l'istruzione corrente dalla memoria all'indirizzo specificato nel **Program Counter (PC)**.
- **Decode:** identifica il tipo di istruzione (ad esempio somma, load, store).
- **Execute:** esegue l'operazione richiesta dall'istruzione.
- **Memory:** interagisce con la memoria se l'istruzione lo richiede (ad esempio in caso di load o store).
- **Write Back:** scrive i risultati nel banco dei registri.
- **Incremento PC:** Il PC viene incrementato di 4 byte per puntare alla successiva istruzione, in ARMv7 ogni istruzione è 32 bit ma la cella di memoria è 1 byte, quindi ne salta 4.

## 1.3 Architettura RISC e CISC

Entrambe le architetture sono potenti allo stesso modo, ovvero hanno la stessa espressività.

- **SISC (Reduced Instruction Set Computer):** utilizza istruzioni semplici ed eseguite in un singolo ciclo di clock, favorendo la pipeline. Hanno un circuito semplice e veloce sia per la decodifica che per l'esecuzione.
- **CISC (Complex Instruction Set Computer):** offre istruzioni più complesse che possono richiedere più cicli di clock, riducendo la necessità di combinare più istruzioni semplici.

## 1.4 Linguaggio Macchina

Il linguaggio macchina è il linguaggio nativo del processore, cioè quello che la CPU è in grado di eseguire direttamente. È costituito da sequenze di bit (0 e 1) che rappresentano operazioni specifiche e sono progettate per essere comprese dalla CPU senza necessità di traduzioni aggiuntive. Ogni tipo di CPU (o architettura) ha il proprio set di istruzioni di linguaggio macchina; quindi, il codice in linguaggio macchina è generalmente non portabile tra diverse architetture.

### 1.4.1 Caratteristiche principali:

- **Binario:** ogni istruzione è rappresentata da combinazioni di 0 e 1.

- **Dipendente dall'architettura:** il linguaggio macchina è diverso per ogni tipo di CPU (x86, ARM, MIPS, ecc.).
- **Direttamente eseguibile:** la CPU legge le istruzioni dalla memoria e le esegue senza bisogno di traduzione.
- **Struttura di un'istruzione:** ogni istruzione è composta da:
  - **Opcode:** indica l'operazione da eseguire (es. add, sub, load, store)
  - **Operandi:** specificano i dati su cui operare o dove salvarli (registri o indirizzi di memoria)
- **Efficienza e Velocità:** Il linguaggio macchina è il più efficiente in termini di esecuzione, poiché non richiede alcuna conversione aggiuntiva per essere eseguito dalla CPU. Si possono inoltre utilizzare i registri in modo esplicito; i registri infatti sono il sistema di memorizzazione più veloce di tutto il sistema per cui c'è uno scambio continuo tra memoria e registri.

## 1.5 Architettura ARMv7:

L'ARMv7 è un'architettura RISC, progettata per massimizzare l'efficienza energetica e la velocità di esecuzione delle istruzioni. La CPU ARMv7 utilizza un set di **16 registri architetturali**, denominati da **R0** a **R15**, ognuno a 32 bit.

Table 1: Registri principali dell'architettura ARMv7

| Registro        | Nome                 | Descrizione                                             |
|-----------------|----------------------|---------------------------------------------------------|
| <b>R0 – R12</b> | Generali             | Registri generici per operazioni aritmetiche e logiche. |
| <b>R13</b>      | SP (Stack Pointer)   | Punta alla cima dello stack.                            |
| <b>R14</b>      | LR (Link Register)   | Contiene l'indirizzo di ritorno da una funzione.        |
| <b>R15</b>      | PC (Program Counter) | Contiene l'indirizzo dell'istruzione corrente.          |

Un'istruzione è composta da **Operatore**, **Destinazione**, **Sorgente1** e **Sorgente2**. La **Destinazione** e la **Sorgente1** sono indirizzi di registri: la loro dimensione è di 4 bit, sufficiente ad indirizzare i 16 (o 24) registri disponibili. La **Sorgente2** può essere sia un riferimento a un registro sia un **immediato**, ovvero una costante.

## 1.6 Linguaggio Assembly:

Il linguaggio assembly è la forma leggibile dall'uomo del linguaggio nativo del calcolatore (linguaggio macchina). Il linguaggio assembly, seppur di basso livello, utilizza simboli (mnemonici) per rappresentare le istruzioni, risultando più leggibile.

### 1.6.1 Caratteristiche principali:

- Ogni **istruzione** assembly rappresenta un'operazione elementare: somma, salto, caricamento di un valore in un registro, ecc.
- Utilizza **mnemonici**, cioè abbreviazioni leggibili per gli esseri umani, come MOV (move), ADD (addizione), SUB (sottrazione), CMP (confronto), B (branch).
- Lavora direttamente con i registri del processore (es. R0, R1, PC, SP, ecc.).
- Ogni architettura (come ARM, x86, MIPS, RISC-V) ha il proprio set di istruzioni, quindi il codice Assembly non è portabile.

Table 2: Istruzioni aritmetiche principali dell'architettura ARMv7

| Operazione | Descrizione                                                                    | Esempio                              |
|------------|--------------------------------------------------------------------------------|--------------------------------------|
| <b>ADD</b> | Somma tra due registri e salva il risultato nel registro di destinazione.      | ADD R0, R1, R2 — $R0 = R1 + R2$      |
| <b>SUB</b> | Sottrae il contenuto di due registri e memorizza nel registro di destinazione. | SUB R0, R1, R2 — $R0 = R1 - R2$      |
| <b>MUL</b> | Moltiplica i valori di due registri e memorizza nel registro di destinazione.  | MUL R0, R1, R2 — $R0 = R1 \times R2$ |
| <b>DIV</b> | Divide il contenuto di un registro per un altro (divisione intera).            | UDIV R0, R1, R2 — $R0 = R1 \div R2$  |

Table 3: Istruzioni logiche e di shift dell'architettura ARMv7

| Operazione | Descrizione                                                                                                    | Esempio                                        |
|------------|----------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| <b>AND</b> | Esegue un'operazione logica AND bit a bit tra due registri e salva il risultato nel registro di destinazione.  | AND R0, R1, R2 — $R0 = R1 \& R2$               |
| <b>ORR</b> | Esegue un'operazione logica OR bit a bit tra due registri e salva il risultato nel registro di destinazione.   | ORR R0, R1, R2 — $R0 = R1    R2$               |
| <b>LSL</b> | Esegue uno <b>shift logico a sinistra</b> dei bit nel registro sorgente di un numero specificato di posizioni. | LSL R0, R1, #2 — $R0 = R1 \ll 2$               |
| <b>LSR</b> | Esegue uno <b>shift logico a destra</b> riempiendo con zeri.                                                   | LSR R0, R1, #2 — $R0 = R1 \gg 2$ (senza segno) |
| <b>ASR</b> | Esegue uno <b>shift aritmetico a destra</b> , mantenendo il segno del numero.                                  | ASR R0, R1, #2 — $R0 = R1 \gg 2$ (con segno)   |
| <b>ROR</b> | Esegue una <b>rotazione a destra</b> dei bit del registro sorgente.                                            | ROR R0, R1, #4 — rotazione di 4 bit a destra   |

### 1.6.2 Registri di uso speciale

Nell'architettura **ARMv7**, oltre ai registri generali (**R0–R12**), esistono tre **registri di uso speciale** fondamentali:

- **PC (Program Counter) = R15**

Contiene l'indirizzo dell'istruzione corrente o della prossima da eseguire. Durante il ciclo di esecuzione, il PC viene aggiornato automaticamente (ad esempio, incrementato di 4 byte dopo ogni istruzione). In pratica, indica *dove si trova* il processore nel programma.

- **LR (Link Register) = R14**

Memorizza l'indirizzo di ritorno da una funzione o subroutine. Quando si effettua una chiamata di funzione (BL in ARM), l'indirizzo dell'istruzione successiva viene salvato in LR, così che al termine della funzione la CPU possa ritornare al punto corretto.

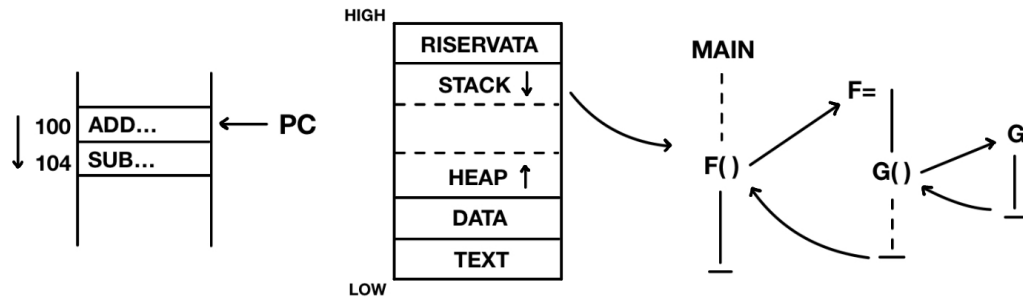
- **SP (Stack Pointer) = R13**

Punta alla cima dello **stack**, una sezione di memoria utilizzata per memorizzare variabili locali, indirizzi di ritorno e registri temporanei. Lo stack cresce verso indirizzi di memoria più bassi (*dalla basso verso l'alto* nel diagramma).

La memoria del processo è organizzata in più sezioni:

- **Text:** contiene il codice eseguibile del programma.

- **Data:** area che memorizza variabili globali e statiche.
- **Heap:** cresce verso indirizzi di memoria più alti, utilizzato per l'allocazione dinamica.
- **Stack:** cresce verso indirizzi di memoria più bassi, utilizzato per gestire chiamate di funzione e variabili locali.



### 1.6.3 Salto condizionato e incondizionato

Un **salto** (in inglese jump o **branch**) è un'istruzione che modifica il normale flusso di esecuzione di un programma. Normalmente, un processore esegue le istruzioni una dopo l'altra, in ordine di memoria. Il salto, invece, cambia questo ordine:

- Dice al processore di **andare a eseguire un'altra istruzione** situata in un indirizzo diverso del programma.
- Questo nuovo indirizzo può essere più **avanti** (salto in avanti) o più **indietro** (salto all'indietro, usato nei cicli).
- Esistono due tipologie di salti:
  - **Salto incondizionato:** viene sempre eseguito, indipendentemente da qualsiasi condizione. Il processore aggiorna direttamente il *Program Counter* con il nuovo indirizzo di destinazione.
  - **Salto condizionato:** viene eseguito solo se una determinata *condizione logica* è vera (ad esempio, se un registro vale zero o se un confronto restituisce “uguale”). In caso contrario, l'esecuzione prosegue normalmente.

#### 1.6.4 L'istruzione "Compare"

In Assembly, l'istruzione **CMP** (abbreviazione di compare, “confronta”) serve per **confrontare due valori**, ma **non memorizza il risultato**: modifica solo i flag nel registro dei flag, in base alla differenza tra gli operandi.

Table 4: Principali **flag** modificati da **CMP**

| Flag                      | Significato              | Quando è impostato (1)                                 |
|---------------------------|--------------------------|--------------------------------------------------------|
| <b>ZF</b> (Zero Flag)     | Il risultato è zero.     | $A == B$                                               |
| <b>SF</b> (Sign Flag)     | Il risultato è negativo. | Il bit più significativo del risultato è 1             |
| <b>CF</b> (Carry Flag)    | C'è stato un "prestito". | $A < B$ (per numeri senza segno)                       |
| <b>OF</b> (Overflow Flag) | Overflow aritmetico.     | La differenza non entra nel range ( <b>con segno</b> ) |

### Esercizio - calcolo del fattoriale:

```
.text
.global fact
.type fact, %function

fact:
    MOV R1, R0        @ Copia il parametro in R1
    MOV R0, #1        @ Inizializza il risultato a 1

start:
    CMP R1, #0        @ Confronta R1 con 0
    BEQ fine          @ Se uguale, vai a fine
    MUL R0, R0, R1     @ R0 = R0 * R1
    SUB R1, R1, #1     @ Decrementa R1
    B start           @ Ripeti il ciclo

fine:
    MOV PC, LR        @ Ritorna al chiamante
```